



南開大學
Nankai University

《并行与分布式程序设计》实验报告

(2025~2026 学年第一学期)

实验名称: MPI 编程练习

姓 名: 李宗杭

学 号: 2311451

专 业: 软件工程

2025 年 12 月 21 日

目 录

1	梯形积分法的 MPI 版本实现	2
1.1	问题描述	2
1.2	算法设计与实现	2
1.2.1	静态分配	2
1.2.2	动态分配	2
1.3	实验及结果分析	4
1.3.1	测试方法	4
1.3.2	结果分析	5
1.3.3	实际运行结果	6
2	”多个数组排序”的 MPI 版本实现	8
2.1	问题描述	8
2.2	算法设计与实现	8
2.2.1	静态分配	8
2.2.2	动态分配	9
2.2.3	初始化	10
2.3	实验及结果分析	11
2.3.1	测试方法	11
2.3.2	结果分析	11
2.3.3	实际运行结果	14
3	矩阵乘法的 MPI 版本实现	15
3.1	问题描述	15
3.2	算法设计与实现	15
3.2.1	静态分配	15
3.2.2	动态分配	16
3.2.3	SIMD	17
3.3	实验及结果分析	18
3.3.1	测试方法	18
3.3.2	结果分析	18
3.3.3	实际运行结果	20

1 梯形积分法的 MPI 版本实现

1.1 问题描述

实现第 5 章课件中的梯形积分法的 MPI 编程熟悉并掌握 MPI 编程方法，规模自行设定，可探讨不同规模对不同实现方式的影响。

1.2 算法设计与实现

1.2.1 静态分配

静态分配任务，每个进程根据总进程数 size 和自己的编号 my_rank，自行计算获取自己需处理的部分，随后进行计算。最终使用 MPI_Reduce 归约结果。显然这是一个对等模型，各个进程功能相当，任务自行获取而非统一分配。

```
1 // 静态任务分配-对等模型
2 void integral_mpi_static(double a, double b, int n, double& result) {
3     double h = (b - a) / n;
4     double local_sum = 0.0;
5     int local_n = (n + size - 1) / size;
6     int left = my_rank * local_n + 1;
7     int right = min(n - 1, left + local_n - 1);
8
9     for (int i = left; i <= right; i++) {
10         double x = a + i * h;
11         local_sum += f(x);
12     }
13
14     double global_sum = 0.0;
15     double init_val = (f(a) + f(b)) / 2.0;
16     MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, 0,
17               ↪ MPI_COMM_WORLD);
18     if (my_rank == 0) {
19         result = (init_val + global_sum) * h;
20     }
```

1.2.2 动态分配

采用主从模型，由 0 号进程来统筹分配每个从进程需处理的任务，使用基础的 MPI_Send 和 MPI_Recv 进行通信。先进行一轮初始分配，随后从进程进行计算，计算结束后把局部结果发给主进程，主进程在收到局部结果后为该进程分配新的任务。

设置 chunk 参数动态调整每次分配的任务量。

```
1 // 动态任务分配-主从模型
2 void integral_mpi_dynamic(double a, double b, int n, int chunk, double& result)
3 {
4     double h = (b - a) / n;
5     MPI_Status status;
6
7     if (my_rank == 0) {
8         double global_sum = 0.0;
9         int task = 0;          // 待分配任务
10        int finished = 1;      // 已完成进程数
11        double recv_sum = 0.0;
12
13        // 分配初始任务
14        for (int i = 1; i < size; i++) {
15            if (task < n) {
16                MPI_Send(&task, 1, MPI_INT, i, task, MPI_COMM_WORLD);
17                task += chunk;
18            }
19            else { // 若初始阶段就分配完
20                MPI_Send(&task, 1, MPI_INT, i, n, MPI_COMM_WORLD); // 发送结束信
21                号
22                finished++;
23            }
24        }
25
26        // 接收结果并分配新任务
27        while (finished < size) {
28            MPI_Recv(&recv_sum, 1, MPI_DOUBLE, MPI_ANY_SOURCE, MPI_ANY_TAG,
29                MPI_COMM_WORLD, &status);
30            if (status.MPI_TAG < n) {
31                global_sum += recv_sum;
32            }
33
34            if (task < n) {
35                MPI_Send(&task, 1, MPI_INT, status.MPI_SOURCE, task,
36                MPI_COMM_WORLD);
37                task += chunk;
38            }
39            else { // 发送结束信号
40                MPI_Send(&task, 1, MPI_INT, status.MPI_SOURCE, n,
41                MPI_COMM_WORLD);
```

```
37         finished++;
38     }
39 }
40
41 // 计算最终结果
42 double init_val = (f(a) + f(b)) / 2.0;
43 result = (init_val + global_sum) * h;
44 }
45 else {
46     int task = 0;
47     double local_sum = 0.0;
48
49     while (true) {
50         MPI_Recv(&task, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD,
51             ↳ &status);
52         if (status.MPI_TAG >= n) break;
53
54         int local_right = min(task + chunk - 1, n - 1);
55         local_sum = 0.0;
56         for (int i = task; i <= local_right; i++) {
57             double x = a + i * h;
58             local_sum += f(x);
59         }
60         MPI_Send(&local_sum, 1, MPI_DOUBLE, 0, status.MPI_TAG,
61             ↳ MPI_COMM_WORLD);
62     }
63 }
```

1.3 实验及结果分析

1.3.1 测试方法

测试不同分块大小及不同规模下各任务分配方式的运行情况。

- 被积分函数 $f(x)$: $x^2 + 2x + 1$;
- 积分区间 $[a, b]$: $[0, 10]$;

为减少测试时间，其中不受变量影响的方法仅测试一次，如测试不同分块大小时，静态分配与细粒度动态分配仅测试一次。

由于此次实验切换到了 linux 系统，之前实验的 QueryPerformance 方法不可用，故采用 MPI 标准提供的 MPI_Wtime 方法进行计时。

每种情况每个方法测试 10 次取平均值，获取稳定结果，降低误差。

1.3.2 结果分析

(1) 不同动态分块下各个方法的运行时间对比

如图1.1，细粒度动态分配的运行时间远高于静态分配和粗粒度动态，这是由于对于每一个单位的任务，它都要分配和接受一次，通信耗时远高于计算耗时。

随着分块变大，粗粒度耗时持续减小，单次分配任务量的增大极大减少了通信次数，且充分发挥了并行计算的优势。

然而，即使分块增大到了 4096（本次测试最大值），粗粒度耗时仍是静态分配的十倍左右，这是对于梯形积分法每个单位的任务计算量一致，且在静态平均分配任务的情况下，基本不存在负载不均的情况，且计算过程中完全不需要进行通信，因此效率最高。

动态分块	静态	动态	粗粒度
2	0.4945	723.3774	367.4563
4	0.4945	723.3774	183.6012
8	0.4945	723.3774	92.6346
16	0.4945	723.3774	49.0108
32	0.4945	723.3774	26.4109
64	0.4945	723.3774	14.9118
128	0.4945	723.3774	10.7490
256	0.4945	723.3774	6.9297
512	0.4945	723.3774	5.4241
1024	0.4945	723.3774	5.6265
2048	0.4945	723.3774	3.2058
4096	0.4945	723.3774	3.3405

表 1: 不同动态分块下运行时间（单位：ms）

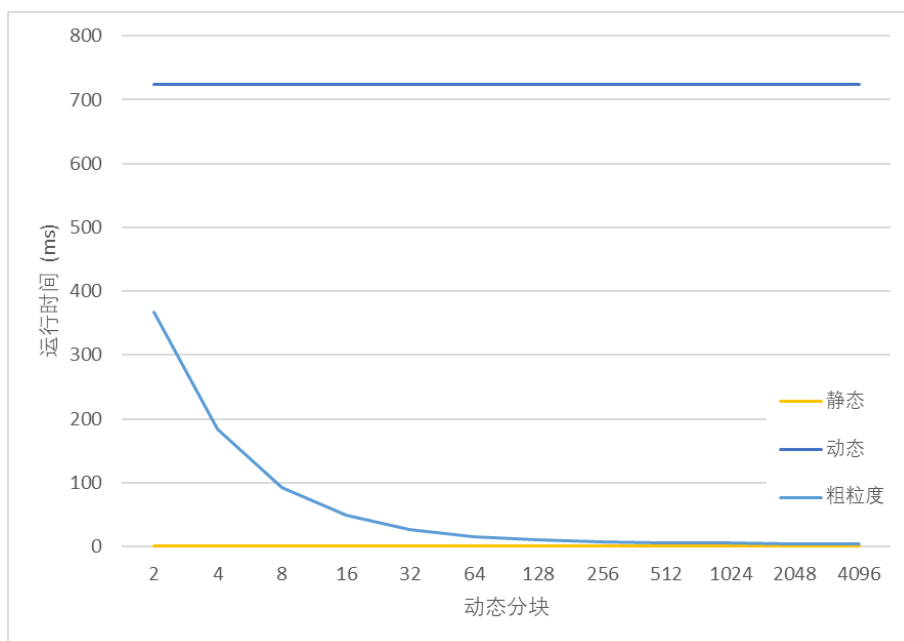


图 1.1: 不同动态分块下运行时间

(2) 不同规模下各个方法的运行时间对比

如图1.2, 由于细粒度动态分配耗时太大, 这里仅展示静态分配和粗粒度动态, 随着规模的增长, 可以看到每种方法的耗时均成线性增长。

规模	静态	动态	粗粒度
10000	0.0246	8.5062	0.2112
100000	0.0699	72.5225	0.3651
1000000	0.4942	717.7627	2.6615
10000000	4.6578	7201.6470	26.8720

表 2: 不同规模下运行时间 (单位: ms)

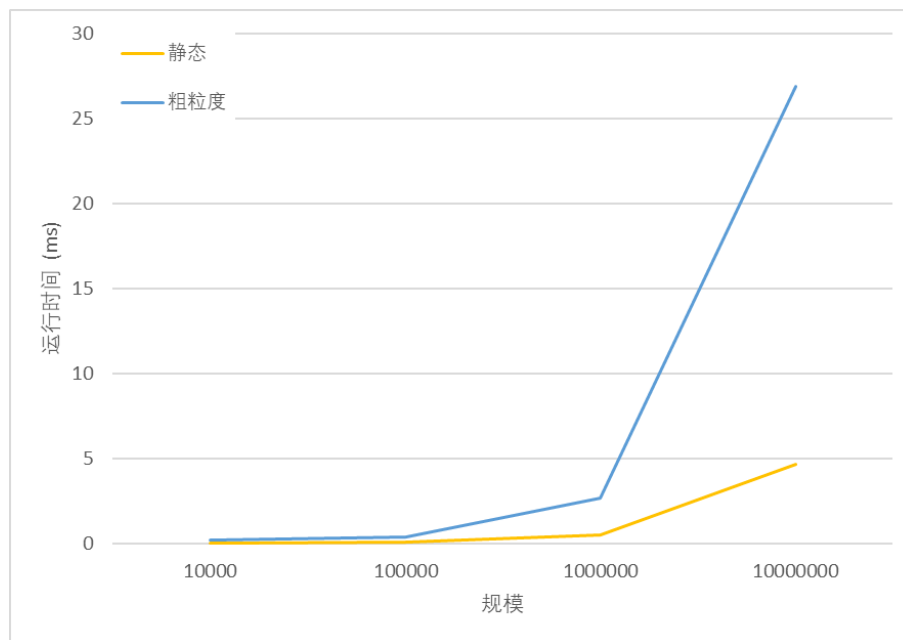


图 1.2: 不同规模下运行时间

1.3.3 实际运行结果

```

[liozhang@ecs-hw-0001 mpi_homework]$ make
mpicxx -O3 -Wall -o mpi_homework main.cpp integral_mpi.cpp sort_mpi.cpp matrix_multiply.cpp matrix_multiply_mpi.cpp utils.cpp -lm
[liozhang@ecs-hw-0001 mpi_homework]$ scp -r /home/liozhang/mpi_homework liozhang@ecs-hw-0002:/home/liozhang/
Authorized users only. All activities may be monitored and reported.
main.cpp                                100% 1507  4.0MB/s  00:00
matrix_multiply.cpp                     100% 3593 11.0MB/s  00:00
sort_mpi.h                              100% 422  1.0MB/s  00:00
matrix_multiply.h                        100% 399  1.2MB/s  00:00
matrix_multiply_mpi.cpp                  100% 7027 23.2MB/s  00:00
integral_mpi.h                           100% 469  1.0MB/s  00:00
mpi_homework                             100% 74KB  95.0MB/s  00:00
matrix_multiply_mpi.h                    100% 432  1.6MB/s  00:00
integral_mpi.cpp                         100% 5534 19.7MB/s  00:00
mainfile                                 100% 288  1.2MB/s  00:00
utils.h                                  100% 568  1.0MB/s  00:00
utils.cpp                                 100% 199  835.3KB/s  00:00
sort_mpi.cpp                             100% 6127 21.1MB/s  00:00
config                                   100% 45  42.2KB/s  00:00
[liozhang@ecs-hw-0001 mpi_homework]$ scp -r /home/liozhang/mpi_homework liozhang@ecs-hw-0002:/home/liozhang/
Authorized users only. All activities may be monitored and reported.
main.cpp                                100% 1507  3.4MB/s  00:00
matrix_multiply.cpp                     100% 3593 11.2MB/s  00:00
sort_mpi.h                              100% 422  1.5MB/s  00:00
matrix_multiply.h                        100% 399  1.2MB/s  00:00
matrix_multiply_mpi.cpp                  100% 7027 22.8MB/s  00:00
integral_mpi.h                           100% 469  1.0MB/s  00:00
mpi_homework                             100% 74KB  99.9MB/s  00:00
matrix_multiply_mpi.h                    100% 432  1.6MB/s  00:00
integral_mpi.cpp                         100% 5534 19.8MB/s  00:00
mainfile                                 100% 288  893.3KB/s  00:00
utils.h                                  100% 568  1.0MB/s  00:00
utils.cpp                                 100% 199  836.6KB/s  00:00
sort_mpi.cpp                             100% 6127 19.2MB/s  00:00
config                                   100% 45 184.9KB/s  00:00

```

```
[lizonghang@ecs-hw-0001 mpi_homework]$ mplexec -n 6 -f config ./mpi_homework
Authorized users only. All activities may be monitored and reported.
Authorized users only. All activities may be monitored and reported.
=====梯形积分法=====
计算结果 443.333333 443.333343 443.333343

动态分块 静态      动态      粗粒度
2          0.4945      723.3774    367.4563
4          0.4945      723.3774    183.6012
8          0.4945      723.3774    92.6346
16         0.4945      723.3774    49.0108
32         0.4945      723.3774    26.4109
64         0.4945      723.3774    14.9118
128        0.4945      723.3774    10.7490
256        0.4945      723.3774    6.9297
512        0.4945      723.3774    5.4241
1024       0.4945      723.3774    5.6265
2048       0.4945      723.3774    3.2058
4096       0.4945      723.3774    3.3405

规模      静态      动态      粗粒度
10000     0.0246     8.5062     0.2112
100000    0.0699     72.5225    0.3651
1000000   0.4942     717.7627   2.6615
10000000  4.6578     7201.6470  26.8720
```

实际运行结果中还展示了计算结果的正确性，可以看到运算结果整体是正确的，仅存在浮点运算精度导致的微小偏差。

2 ”多个数组排序”的 MPI 版本实现

2.1 问题描述

对于课件中“多个数组排序”的任务不均衡案例进行 MPI 编程实现，规模可自己设定、调整。

2.2 算法设计与实现

2.2.1 静态分配

这里的静态分配与梯形积分法不同，采用主从模型，由 0 号进程负责分配任务和汇总结果。同时为了提高效率，为 0 号进程也分配一份任务。

另外，这里的数组采用 vector，而多层 vector 的内层 vector 之间地址是不连续的，因此内层 vector 需要逐条发送和接收。这点将在矩阵乘法实验中得到优化体现。

```
1 // 静态任务分配-主从模型
2 void sort_mpi_static(int arr_num, int arr_len) {
3     int local_num = (arr_num + size - 1) / size;
4     int left = my_rank * local_num;
5     int right = min(arr_num, left + local_num);
6
7     for (int i = left; i < right; i++) {
8         stable_sort(arrays[i].begin(), arrays[i].end());
9     }
10
11     if (my_rank == 0) {
12         for (int i = 1; i < size; i++) {
13             int slave_left = i * local_num;
14             int slave_right = min(arr_num, slave_left + local_num);
15             // vector 地址不连续，需逐条接收内层数组
16             for (int j = slave_left; j < slave_right; j++) {
17                 MPI_Recv(arrays[j].data(), arr_len, MPI_INT, i, 0,
18                     ↪ MPI_COMM_WORLD, MPI_STATUS_IGNORE);
19             }
20         }
21     } else {
22
23         for (int i = left; i < right; i++) {
24             MPI_Send(arrays[i].data(), arr_len, MPI_INT, 0, 0, MPI_COMM_WORLD);
25         }
26     }
27 }
```

2.2.2 动态分配

动态分配方式与梯形积分法基本一致，chunk 参数可控。

```
1 // 动态任务分配-主从模型
2 void sort_mpi_dynamic(int arr_num, int arr_len, int chunk) {
3     MPI_Status status;
4
5     if (my_rank == 0) {
6         int task = 0;
7         int finished = 1;
8
9         for (int i = 1; i < size; i++) {
10             if (task < arr_num) {
11                 MPI_Send(&task, 1, MPI_INT, i, task, MPI_COMM_WORLD);
12                 task += chunk;
13             }
14             else { // 若初始阶段就分配完
15                 MPI_Send(&task, 1, MPI_INT, i, arr_num, MPI_COMM_WORLD); // 发送
16                     ↪ 结束信号
17                 finished++;
18             }
19
20             while (finished < size) {
21                 int task_id;
22                 // 接收任务 id
23                 MPI_Recv(&task_id, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG,
24                     ↪ MPI_COMM_WORLD, &status);
25                 if (status.MPI_TAG < arr_num) {
26                     // 接收排序结果
27                     for (int i = 0; i < min(chunk, arr_num - task_id); i++) {
28                         MPI_Recv(arrays[task_id + i].data(), arr_len, MPI_INT,
29                             ↪ status.MPI_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD,
30                             ↪ &status);
31                     }
32                 }
33
34                 if (task < arr_num) {
35                     MPI_Send(&task, 1, MPI_INT, status.MPI_SOURCE, task,
36                         ↪ MPI_COMM_WORLD);
37                     task += chunk;
38                 }
39             }
40         }
41     }
42 }
```

```

35         else { // 发送结束信号
36             MPI_Send(&task, 1, MPI_INT, status.MPI_SOURCE, arr_num,
37                     ↪ MPI_COMM_WORLD);
38             finished++;
39         }
40     }
41     else {
42         int task = 0;
43         while (true) {
44             MPI_Recv(&task, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD,
45                     ↪ &status);
46             if (status.MPI_TAG >= arr_num) break;
47
48             for (int i = 0; i < chunk && task + i < arr_num; i++) {
49                 sort(arrays[task + i].begin(), arrays[task + i].end());
50             }
51             MPI_Send(&task, 1, MPI_INT, 0, status.MPI_TAG, MPI_COMM_WORLD);
52
53             for (int i = 0; i < min(chunk, arr_num - task); i++) {
54                 MPI_Send(arrays[task + i].data(), arr_len, MPI_INT, 0,
55                         ↪ status.MPI_TAG, MPI_COMM_WORLD);
56             }
57         }
58     }
59 }

```

2.2.3 初始化

将对数组的初始化与广播初始化结果放在了计时函数中，每轮测试都要重新进行初始化以加强随机性。

```

1 // 计时函数
2 double measure_time_sort(int type, int arr_num, int arr_len, int chunk, int
3   ↪ repeat) {
4     double start_time, end_time, total_time = 0.0;
5     for (int i = 0; i < repeat; i++) {
6         init_arrays(arr_num, arr_len);
7         start_time = MPI_Wtime();
8         for (int i = 0; i < arr_num; i++) {
9             MPI_Bcast(arrays[i].data(), arr_len, MPI_INT, 0, MPI_COMM_WORLD);
10        }
11    }
12    end_time = MPI_Wtime();
13    total_time = end_time - start_time;
14    return total_time;
15 }

```

```
10         if (type == 0) {
11             sort_mpi_static(arr_num, arr_len);
12         }
13         else if (type == 1) {
14             sort_mpi_dynamic(arr_num, arr_len, chunk);
15         }
16         end_time = MPI_Wtime();
17         total_time += (end_time - start_time);
18     }
19     return (total_time / repeat) * 1000;
20 }
```

2.3 实验及结果分析

2.3.1 测试方法

测试不同分块大小及不同规模下各任务分配方式的运行情况, 不受变量影响的方法仅测试一次, 采用 MPI 标准 MPI_Wtime 方法进行计时, 每种情况每个方法测试 5 次取平均值, 获取稳定结果, 降低误差。

2.3.2 结果分析

(1) 不同动态分块下各个方法的运行时间对比

如图2.3, 首先可以看到静态分配与细粒度动态耗时相差不大, 动态分配略优于静态分配。这是因为“多个数组排序”问题不同于“梯形积分法”问题, 该问题手动模拟了负载不均衡的场景, 因此单纯静态分配的方法就会出现负载不均的情况, 而细粒度动态分配虽然可以避免负载不均, 但是由于每次通信只传输一个单位的任务, 通信次数过多, 通信耗时过大, 与其负载均衡带来的优势抵消了一部分, 因此在本次实验中并没有比静态分配好多少。

再来看粗粒度动态分配方式, 整体而言其耗时随着分块的增大而增大。当分块较小时, 其耗时优于静态和细粒度动态, 因为它既能缓解负载不均, 同时单次分配任务量变大而通信次数减少, 有效的平衡了负载均衡的优势和通信带来的开销。但当分块变的过大时, 负载不均重新出现, 且又比静态分配多了通信开销, 因此反而不如单纯静态和细粒度动态。

动态分块	静态	动态	粗粒度
2	2106.0911	2066.5121	1991.1053
4	2106.0911	2066.5121	2028.1435
8	2106.0911	2066.5121	1984.3147
16	2106.0911	2066.5121	2010.1954
32	2106.0911	2066.5121	2004.7035
64	2106.0911	2066.5121	2019.7216
128	2106.0911	2066.5121	2002.5087
256	2106.0911	2066.5121	2030.9106
512	2106.0911	2066.5121	2007.0498
1024	2106.0911	2066.5121	2033.5167
2048	2106.0911	2066.5121	2219.7336

表 3: 不同动态分块下运行时间 (单位: ms)

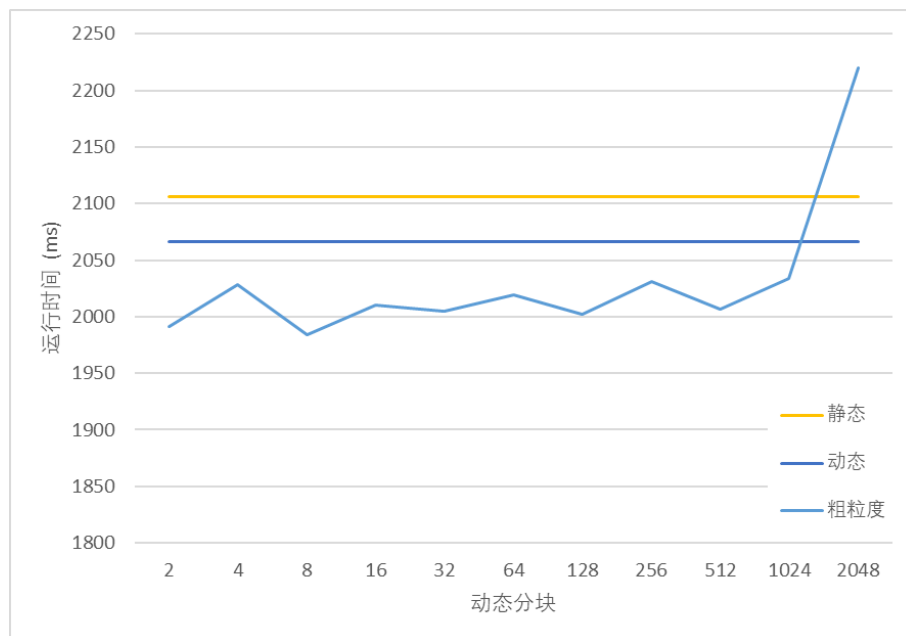


图 2.3: 不同动态分块下运行时间

(2) 不同规模下各个方法的运行时间对比

根据分块测试结果, 选取一个较优的分块数 4 进行规模测试。随着规模的增大, 各方法耗时呈线性增长。且平衡了负载均衡和通信开销的粗粒度动态优于通信开销较大的细粒度动态优于负载不均的静态。

规模	静态	动态	粗粒度
2500	364.4445	557.4368	464.1806
5000	1020.3602	1041.5573	980.1713
7500	1547.0609	1522.2633	1493.9779
10000	2219.0715	2089.2060	1997.9702
12500	2929.5900	2808.7828	2679.0869
15000	3613.7191	3530.6737	3299.8134
17500	4429.2459	4265.1381	3976.7232
20000	5380.8628	5016.3419	4697.1131

表 4: 不同规模下运行时间 (单位: ms)

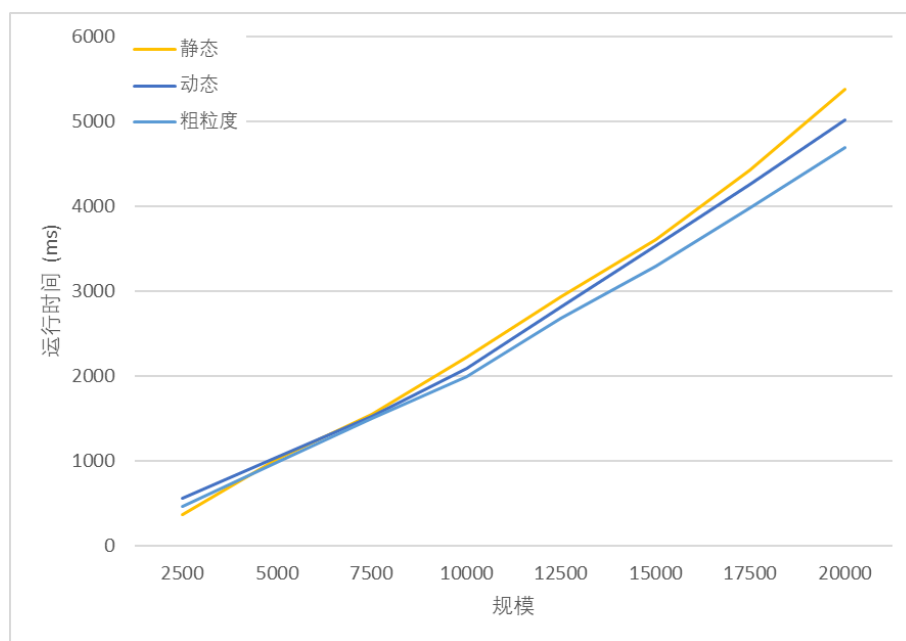


图 2.4: 不同规模下运行时间

2.3.3 实际运行结果

=====多个数组排序=====			
动态分块	静态	动态	粗粒度
2	2106.0911	2066.5121	1991.1053
4	2106.0911	2066.5121	2028.1435
8	2106.0911	2066.5121	1984.3147
16	2106.0911	2066.5121	2010.1954
32	2106.0911	2066.5121	2004.7035
64	2106.0911	2066.5121	2019.7216
128	2106.0911	2066.5121	2002.5087
256	2106.0911	2066.5121	2030.9106
512	2106.0911	2066.5121	2007.0498
1024	2106.0911	2066.5121	2033.5167
2048	2106.0911	2066.5121	2219.7336
规模	静态	动态	粗粒度
2500	364.4445	557.4368	464.1806
5000	1020.3602	1041.5573	980.1713
7500	1547.0609	1522.2633	1493.9779
10000	2219.0715	2089.2060	1997.9702
12500	2929.5900	2808.7828	2679.0869
15000	3613.7191	3530.6737	3299.8134
17500	4429.2459	4265.1381	3976.7232
20000	5380.8628	5016.3419	4697.1131

3 矩阵乘法的 MPI 版本实现

3.1 问题描述

实现 MPI 版本的矩阵乘法程序，并和第 2 次作业的四种矩阵乘法程序进行性能比较，规模自己设定。

3.2 算法设计与实现

3.2.1 静态分配

采用主从模型，由 0 号进程负责分配任务和汇总结果。同时为了提高效率，为 0 号进程也分配一份任务。

相比“多个数组排序”，这里对数据结构做了简单的优化，也就是把 vector 换成直接采用 `a[MAX_][MAX_N]`，地址空间连续，因此可以一次传输多条数组，减少了通信次数。

```
1 // 静态任务分配-主从模型
2 void matrix_multiply_mpi_static(int n) {
3     int local_rows = (n + size - 1) / size;
4     int left = my_rank * local_rows;
5     int right = min(n, left + local_rows);
6
7     for (int i = left; i < right; i++) {
8         for (int j = 0; j < n; j++) {
9             float sum = 0.0f;
10            for (int k = 0; k < n; k++) {
11                sum += a[i][k] * b[k][j];
12            }
13            c[i][j] = sum;
14        }
15    }
16
17    if (my_rank == 0) {
18        for (int i = 1; i < size; i++) {
19            int slave_left = i * local_rows;
20            int slave_right = min(n, slave_left + local_rows);
21            MPI_Recv(c[slave_left], n * (slave_right - slave_left), MPI_FLOAT,
22                    i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
23        }
24    } else {
25        MPI_Send(c[left], n * (right - left), MPI_FLOAT, 0, 0, MPI_COMM_WORLD);
26    }
27 }
```


3.2.2 动态分配

同样优化了通信。

```

1 // 动态任务分配-主从模型
2 void matrix_multiply_mpi_dynamic(int n, int chunk) {
3     MPI_Status status;
4
5     if (my_rank == 0) {
6         int task = 0;
7         int finished = 1;
8
9         for (int i = 1; i < size; i++) {
10             if (task < n) {
11                 MPI_Send(&task, 1, MPI_INT, i, task, MPI_COMM_WORLD);
12                 task += chunk;
13             }
14             else { // 若初始阶段就分配完
15                 int term = n;
16                 MPI_Send(&term, 1, MPI_INT, i, n, MPI_COMM_WORLD); // 发送结束信
17                                     ↪ 号
18                 finished++;
19             }
20
21             while (finished < size) {
22                 int task_id;
23                 // 接收任务 id
24                 MPI_Recv(&task_id, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG,
25                                     ↪ MPI_COMM_WORLD, &status);
26                 if (status.MPI_TAG < n) {
27                     // 接收计算结果
28                     MPI_Recv(c[task_id], n * min(chunk, n - task_id), MPI_FLOAT,
29                                     ↪ status.MPI_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
30                 }
31
32                 if (task < n) {
33                     MPI_Send(&task, 1, MPI_INT, status.MPI_SOURCE, task,
34                                     ↪ MPI_COMM_WORLD);
35                     task += chunk;
36                 }
37                 else { // 发送结束信号

```

```

35         MPI_Send(&task, 1, MPI_INT, status.MPI_SOURCE, n,
36                 ↪ MPI_COMM_WORLD);
37         finished++;
38     }
39 }
40 else {
41     int task = 0;
42     while (true) {
43         MPI_Recv(&task, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD,
44                 ↪ &status);
45         if (task >= n) break;
46
47         int task_end = min(task + chunk, n);
48         for (int i = task; i < task_end; i++) {
49             for (int j = 0; j < n; j++) {
50                 float sum = 0.0f;
51                 for (int k = 0; k < n; k++) {
52                     sum += a[i][k] * b[k][j];
53                 }
54                 c[i][j] = sum;
55             }
56         }
57         MPI_Send(&task, 1, MPI_INT, 0, status.MPI_TAG, MPI_COMM_WORLD);
58         MPI_Send(c[task], n * (task_end - task), MPI_FLOAT, 0,
59                 ↪ status.MPI_TAG, MPI_COMM_WORLD);
60     }
61 }

```

3.2.3 SIMD

此次实验引用了第二次作业的代码，测试串行、缓存优化、SIMD 编程与分片策略四种方法并与 MPI 编程进行对比。但由于此次实验换到了 ARM 架构的 linux 系统，因此 x86 SSE 相关的代码需替换成 NEON 指令集的代码。

```

1 void matrix_multiply_neon(int n) {
2     for (int i = 0; i < n; ++i) {
3         for (int j = 0; j < n; ++j) {
4             b_t[i][j] = b[j][i];
5         }
6     }
7 }

```

```

6      }
7      float32x4_t vec_a, vec_b, vec_sum;
8      for (int i = 0; i < n; ++i) {
9          for (int j = 0; j < n; ++j) {
10             c[i][j] = 0.0f;
11             vec_sum = vdupq_n_f32(0.0f);
12             // 批量处理 4 个元素
13             for (int k = n - 4; k > 0; k -= 4) {
14                 vec_a = vld1q_f32(a[i] + k);
15                 vec_b = vld1q_f32(b_t[j] + k);
16                 vec_a = vmulq_f32(vec_a, vec_b);
17                 vec_sum = vaddq_f32(vec_sum, vec_a);
18             }
19             // 水平累加结果
20             float32x2_t sum_low = vadd_f32(vget_low_f32(vec_sum),
21             ↪ vget_high_f32(vec_sum));
22             float32x2_t sum_final = vpadd_f32(sum_low, sum_low);
23             c[i][j] += vget_lane_f32(sum_final, 0);
24             // 处理剩余元素
25             for (int k = n % 4 - 1; k > 0; k--) {
26                 c[i][j] += a[i][k] * b_t[j][k];
27             }
28         }
29     }

```

3.3 实验及结果分析

3.3.1 测试方法

测试不同分块大小及不同规模下各 MPI 任务分配方式的运行情况，并与第二次作业四种方式进行对比，不受变量影响的方法仅测试一次，采用 MPI 标准 MPI_Wtime 方法进行计时，每种情况每个方法测试 10 次取平均值，获取稳定结果，降低误差。

3.3.2 结果分析

(1) 不同动态分块下各个方法的运行时间对比

如图3.5，先看不受 chunk 影响的方法，耗时：静态分配 < 细粒度动态 < NEON < 单纯缓存 < 串行，可见并行编程依旧远优于串行。而分片策略（结合了 NEON）随着分片的增大，其耗时先降后升，在 32 分片时甚至优于 MPI 编程。或许本次实验采取的进程数太少（共六个进程），提升并行计算的进程数量也许能进一步提升并行计算的优势，从而超过分片 + SIMD 策略。

动态分块	串行	缓存	NEON	分片	静态	动态	粗粒度
2	2882.2262	1796.1012	586.8880	2093.4232	513.1594	572.6327	560.2662
4	2882.2262	1796.1012	586.8880	930.8622	513.1594	572.6327	548.8840
8	2882.2262	1796.1012	586.8880	663.6189	513.1594	572.6327	555.0440
16	2882.2262	1796.1012	586.8880	489.2256	513.1594	572.6327	560.4190
32	2882.2262	1796.1012	586.8880	424.1233	513.1594	572.6327	571.9807
64	2882.2262	1796.1012	586.8880	468.1108	513.1594	572.6327	607.0629
128	2882.2262	1796.1012	586.8880	828.1848	513.1594	572.6327	948.2011
256	2882.2262	1796.1012	586.8880	666.0321	513.1594	572.6327	723.8042

表 5: 不同动态分块下运行时间 (单位: ms)

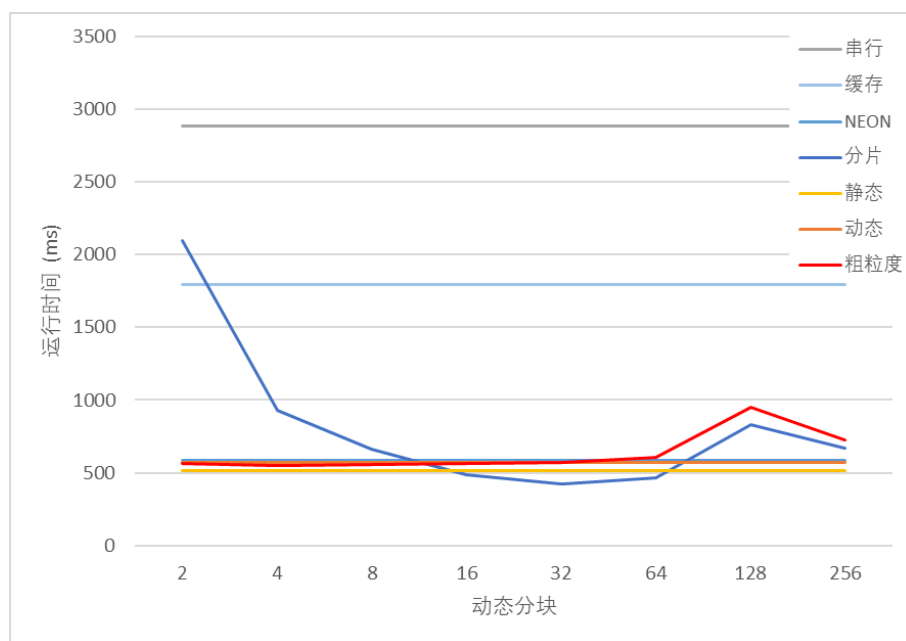


图 3.5: 不同动态分块下运行时间

(2) 不同规模下各个方法的运行时间对比

如图3.6, 选取最优的分片数 32 进行规模测试, 可以看到, 随着规模的增大, 分片 +SIMD 的优势越来越显著。

规模	串行	缓存	NEON	分片	静态	动态	粗粒度
128	2.8824	3.2938	1.2218	0.8543	3.0953	2.7810	1.5331
256	42.1671	27.8692	11.7981	6.9830	10.8029	14.0659	11.5880
384	145.0050	94.8838	40.1492	22.5386	29.1032	42.4127	34.9503
512	356.3984	229.2215	83.7983	52.2391	65.2755	81.0760	72.0270
640	708.0658	442.1490	154.3359	99.9572	127.2592	148.8715	146.8911
768	1237.3252	757.6309	261.2032	174.2523	224.0095	247.9708	246.5141
896	1947.5410	1195.7353	406.1612	273.7749	346.2254	394.0937	376.8168
1024	2887.5899	1812.8831	589.0882	419.0041	522.3040	577.3852	573.4774
1152	4207.9321	2670.7505	829.5092	606.9557	744.6586	807.9391	821.4215
1280	5874.6544	3898.4263	1172.8466	837.2708	1077.4045	1082.4923	1119.8991

表 6: 不同规模下运行时间 (单位: ms)

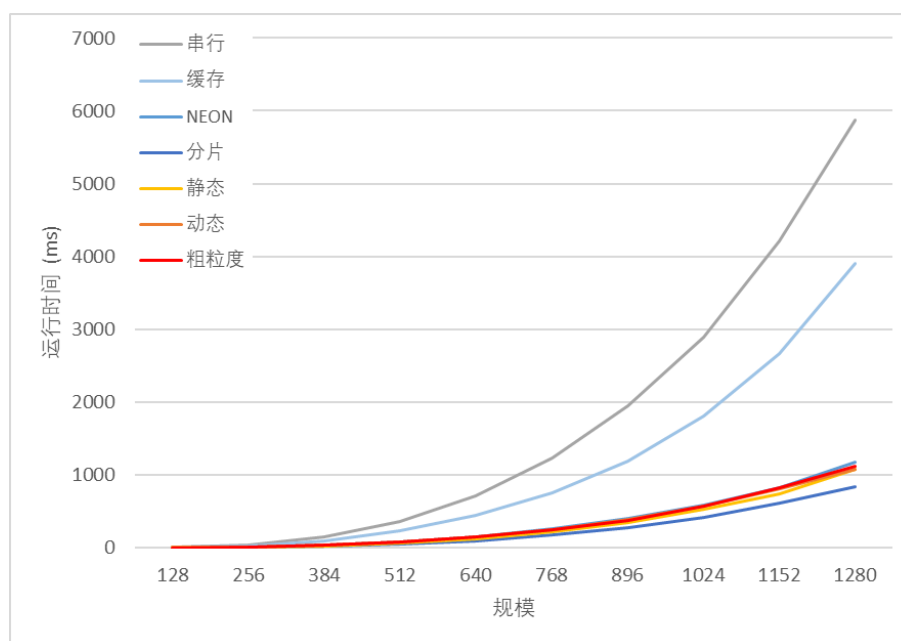


图 3.6: 不同规模下运行时间

3.3.3 实际运行结果

=====矩阵乘法=====								
动态分块	串行	缓存	NEON	分片	静态	动态	粗粒度	
2	2882.2262	1796.1012	586.8880	2093.4232	513.1594	572.6327	560.2662	
4	2882.2262	1796.1012	586.8880	930.8622	513.1594	572.6327	548.8840	
8	2882.2262	1796.1012	586.8880	663.6189	513.1594	572.6327	555.0440	
16	2882.2262	1796.1012	586.8880	489.2256	513.1594	572.6327	560.4190	
32	2882.2262	1796.1012	586.8880	424.1233	513.1594	572.6327	571.9807	
64	2882.2262	1796.1012	586.8880	468.1108	513.1594	572.6327	607.0629	
128	2882.2262	1796.1012	586.8880	828.1848	513.1594	572.6327	948.2011	
256	2882.2262	1796.1012	586.8880	666.0321	513.1594	572.6327	723.8042	
规模	串行	缓存	NEON	分片	静态	动态	粗粒度	
128	2.8824	3.2938	1.2218	0.8543	3.0953	2.7810	1.5331	
256	42.1671	27.8692	11.7981	6.9830	10.8029	14.0659	11.5880	
384	145.0050	94.8838	40.1492	22.5386	29.1032	42.4127	34.9503	
512	356.3984	229.2215	83.7983	52.2391	65.2755	81.0760	72.0270	
640	708.0658	442.1490	154.3359	99.9572	127.2592	148.8715	146.8911	
768	1237.3252	757.6309	261.2032	174.2523	224.0095	247.9708	246.5141	
896	1947.5410	1195.7353	406.1612	273.7749	346.2254	394.0937	376.8168	
1024	2887.5899	1812.8831	589.0882	419.0041	522.3040	577.3852	573.4774	
1152	4207.9321	2670.7505	829.5092	606.9557	744.6586	807.9391	821.4215	
1280	5874.6544	3898.4263	1172.8466	837.2708	1077.4045	1082.4923	1119.8991	